

Esercizio 2 (9 punti) Data una stringa $X = x_1, x_2, \dots, x_n$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i + 1, j - 1) & \text{se } (i < j - 1) \text{ e } (x_i = x_j) \\ \sum_{k=i}^{j-1} (\ell(i, k) + \ell(k + 1, j)) & \text{se } (i < j - 1) \text{ e } (x_i \neq x_j). \end{cases}$$

1. Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{REC_L}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$. (vedo $(1, n)$ e so che la scansione è per diagonale)
2. Si determini la complessità *al caso migliore* $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Soluzione:

1. Pseudocodice:

```
INIT_L(X)
  n <- length(X)          Prendo la lunghezza della stringa iniziale
  if n = 1 then return 1  Similmente, anche n=1 e n=2 vengono dai due casi base (array di 1/2 elementi max),
  if n = 2 then return 2  quindi o ha un elemento oppure ne ha due soli
  for i=1 to n-1 do
    L[i,i] <- 1
    L[i,i+1] <- 2
  L[n,n] <- 1 (ultimo elemento diagonale inizializzato)
  for i=1 to n-2 do
    for j=i+2 to n do
      L[i,j] <- 0
    return REC_L(X,1,n)

REC_L(X,i,j)
  if L[i,j] = 0 then
    if x_i = x_j then L[i,j] <- 2 + REC_L(X,i+1,j-1)
    else for k=i to j-1 do
      L[i,j] <- L[i,j] + REC_L(X,i,k) + REC_L(X,k+1,j)
  return L[i,j]
```

2. Il caso migliore è chiaramente quello in cui tutti i caratteri sono uguali, poiché l'albero della ricorsione in quel caso è unario e i suoi nodi interni corrispondono alle chiamate con indici

2. Il caso migliore è chiaramente quello in cui tutti i caratteri sono uguali, poiché l'albero della ricorsione in quel caso è unario e i suoi nodi interni corrispondono alle chiamate con indici

$(1, n), (2, n - 1), \dots, (k, n - k + 1)$, ognuno associato a un costo unitario. Ci sono al più $\lfloor n/2 \rfloor$ di queste chiamate, e quindi $T_{\text{best}}(n) = O(n)$.

Esercizio 34 L'ufficio postale offre un servizio di ritiro pacchi in sede su prenotazione. Il destinatario, avvisato della presenza del pacco, deve comunicare l'orario preciso al quale si recherà allo

sportello. Sapendo che gli impiegati dedicano a questa mansione turni di un'ora, con inizio in un momento qualsiasi, si chiede di scrivere un algoritmo che individui l'insieme minimo di turni di un'ora sufficienti a soddisfare tutte le richieste. Più in dettaglio, data una sequenza $\vec{r} = r_1, \dots, r_n$ di richieste, dove r_i è l'orario della i -ma prenotazione, si vuole determinare una sequenza di turni $\vec{t} = t_1, \dots, t_k$, con t_j orario di inizio del j -mo turno, che abbia dimensione minima e tale che i turni coprano tutte le richieste.

- i. Formalizzare la nozione di soluzione per il problema e il relativo costo. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.
- ii. Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy `time(R, n)` che dato in input l'array delle richieste `r[1..n]` restituisce una soluzione ottima.
- iii. Valutare la complessità dell'algoritmo.

L'idea è di crearsi un array/sequenza di turni tale che siano con un costo "k" minore possibile.

Soluzione: Una soluzione è una sequenza $\vec{t} = t_1, \dots, t_k$ tale che per ogni $i = 1, \dots, n$ l'istante $r_i \in I(t_j)$ per qualche $j = 1, \dots, k$, dove $I(t_j)$ indica l'intervallo di un'ora con inizio in t_j . Il costo è k .

La sottostruttura ottima vale avendo le richieste e, se abbiamo fatto la scelta minore in partenza tra turni e richieste, di sicuro questa tuttora si mantiene.

Sottostruttura ottima Vale la sottostruttura ottima. Infatti se $\vec{t} = t_1, \dots, t_k$ è ottima per le richieste $\vec{r} = r_1, \dots, r_n$ allora $t_2, \dots, t_k$ è ottima per il sottoproblema $\vec{r}'$ che comprende le richieste r_i tali che $r_i \notin I(t_1)$. Infatti, se ci fosse un insieme di turni di dimensione minore di $k-1$ per servire le richieste in $\vec{r}'$, aggiungendo t_1 otterremmo una soluzione del problema originale $\vec{r}$ migliore di $\vec{t}$.

Scelta greedy Assumiamo che la lista di richieste $\vec{r} = r_1, \dots, r_n$ sia ordinata in modo crescente. La scelta greedy consiste nel considerare come primo turno $t_1 = r_1$.

Esiste sempre una soluzione ottima che la contiene. Infatti se $\vec{t}' = t'_1 \dots t'_k$ è una qualsiasi soluzione ottima e supponiamo che i turni siano ordinati anch'essi in senso crescente, allora certamente $t'_1 \leq r_1$, dato che la prima richiesta deve essere servita. Quindi se sostituiamo t'_1 con $t_1 = r_1$, otteniamo una nuova soluzione (tutte le richieste servite da t'_1 sono anche servite da t_1 !), anch'essa ottima.

Ne segue l'algoritmo che riceve in input l'array delle richieste $r[1..n]$ (che si assume non vuoto), e fornisce in uscita $t[1, k]$.

```
time(r,n)
  t[1] = r[1]
  turni=1
  for i = 2 to n
    if t[turni] < r[i]
      turni++
      t[turni] = r[i]
  return t
```

Salviamo nel vettore dei turni, la prima posizione delle richieste, in maniera tale da avere almeno una richiesta assegnata, ovviamente nel caso in cui non ce ne siano altre. La scelta greedy consiste nel scegliere la prima richiesta ed assegnare le altre, conoscendo già il tempo delle richieste in modo crescente (considerando ovviamente tutti i turni). Quindi, se abbiamo il turno con una richiesta che ci mette poco, la assegniamo e ritorniamo l'array di turni ottimi.

La complessità è $\Theta(n)$.

Domanda A (6 punti) Si determini la soluzione asintotica della seguente equazione di ricorrenza:

$$T(n) = 4T(n/3) + 2n^2 + 1$$

Caso 3 del MT e si risolve con condizione di regolarità.

Soluzione: Rispetto allo schema generale si ha $a = 4$, $b = 3$, $f(n) = 2n^2 + 1$. Si osserva che $\log_b a = \log_3 4 < 2$ quindi $f(n) = \Omega(n^{\log_b a + \epsilon})$ (per $0 < \epsilon \leq 2 - \log_3 4$). In aggiunta vale la condizione di regolarità, ovvero $af(\frac{n}{b}) \leq cf(n)$ per qualche $c < 1$. Infatti $af(\frac{n}{b}) = 4f(\frac{n}{3}) = 4(\frac{2n^2}{9} + 1) \leq 2cn^2 \leq c(2n^2 + 1)$, asintoticamente, quando $4/9 < c < 1$. Più precisamente, per avere

$$4(\frac{2n^2}{9} + 1) \leq 2cn^2$$

Occorre che $c \geq \frac{4}{9} + \frac{2}{n^2}$, che, per $n \geq 2$, si ottiene quando $c \geq \frac{4}{9} + \frac{2}{4} = \frac{15}{18} < 1$.

Esercizio 1 (10 punti) Un array di interi $A[1..n]$ si dice *triangolare* se esiste $q \in [1, n]$ tale che $A[1..q]$ è ordinato in modo crescente e $A[q..n]$ è ordinato in modo decrescente. Realizzare una funzione $\text{maxTr}(A)$ che dato un array triangolare $A[1..n]$, senza elementi ripetuti, ne trova il massimo. Valutarne la complessità.

Soluzione: Si realizza una procedura divide et impera che opera sul sotto-array $A[p, r]$, che inizialmente sarà l'intero array. Se $p = r$, ovvero il sotto-array ha dimensione 1, semplicemente si ritorna l'unico elemento $A[p]$. Altrimenti, si considera il punto intermedio $q = \lfloor p + r \rfloor$. Se $A[q] < A[q + 1]$ significa che la parte $A[p..q]$ è crescente e la parte $A[q+1..r]$ è ancora triangolare, per cui il massimo si troverà in quest'ultima e può essere cercato ricorsivamente. Dualmente se $A[q] > A[q + 1]$ significa che la parte $A[q+1..r]$ è decrescente e la parte $A[p..q]$ è ancora triangolare, per cui il massimo si troverà in quest'ultima.

```

maxTr(A,p,r)
  if p=r
    return A[p]
  else
    q = (p+r)/2
    if A[q]<A[q+1]
      return maxTr(A,q+1,r)
    else
      return maxTr(A,p,q)

```

```

maxTr(A)
  return maxTr(A,1,A.length)

```

Avendo dalla consegna un ragionamento esplicito su due metà distinte di array, allora:

- verifichiamo se abbiamo almeno due metà su cui lavorare (inizio e fine) e se avessimo un solo elemento, lo restituiamo
- prendiamo l'elemento medio e
- se l'elemento medio è minore dell'elemento medio +1, allora spostiamo in avanti l'indice medio (così, il massimo sarà scorrendo avanti sulla seconda partizione)
- se l'elemento medio è maggiore dell'elemento medio +1, allora spostiamo indietro l'indice medio (così, il massimo sarà su "p", quindi sulla prima partizione)

La complessità è espressa dalla ricorrenza

$$T(n) = T(n/2) + c$$

Si può risolvere utilizzando il master theorem, confrontando $n^{\log_b a} = n^{\log_2^2} = n$ con $f(n) = c$. Dato che $f(n) = O(n^{\log_b a - \epsilon}) = O(n^{1-\epsilon})$ per $0 < \epsilon < 1$ si conclude che siamo nel primo caso del teorema che ci dà la soluzione $T(n) = \Theta(\log n)$.

Domanda 28 Sia T un albero binario i cui nodi x hanno i campi $x.left$, $x.right$, $x.key$. L'albero si dice un *sum-heap* se per ogni nodo x , la chiave di x è maggiore o uguale sia alla somma delle chiavi nel sottoalbero sinistro che alla somma delle chiavi nel sottoalbero destro.

Scrivere una funzione `IsSumHeap(T)` che dato in input un albero T verifica se T è un sum-heap e ritorna un corrispondente valore booleano. Valutarne la complessità.

Soluzione: La soluzione può essere

`IsSumHeap(T)`

Chiamata di base per la procedura ricorsiva

`return IsSumHeap-rec(T.root)`

`IsSumHeap-rec(x) // verifica se l'albero radicato in x e' un sum-heap
e ritorna true/false e la somma delle chiavi nel
sottoalbero radicato in x`

`if x = nil`

`return true, 0`

`else`

`isSumHeapL, sumL = IsSumHeap-rec(x.left)`

`isSumHeapR, sumR = IsSumHeap-rec(x.right)`

`return (x.key >= sumL) and (x.key >= sumR),
x.key + sumL + sumR`

Sum-heap: mantiene la proprietà di heap ($\text{parent} \geq \text{left-right}$) e la chiave di "x" è $\geq$ alla somma delle chiavi a sx e dx.

Quindi:

- ricorro a sx, tenendo un booleano di controllo che verifica se la somma dei nodi continua a rispettare la proprietà di heap
- ricorro a dx, tenendo un booleano di controllo che verifica se la somma dei nodi continua a rispettare la proprietà di heap
- ritorna vero se la chiave di "x" è $\geq$ della somma dei nodi di sx e di dx e ritorna la somma di tutti i nodi.

Complessità logaritmica.

Per quanto riguarda la complessità, se l'albero è bilanciato, $T(n) = c + 2T(n/2)$ per un'opportuna costante c e quindi, utilizzando il master theorem, si deduce che la complessità è $\Theta(n)$. Se l'albero non è bilanciato, si può osservare che si tratta di una visita (costo lineare) o più precisamente scrivere la ricorrenza

$$T(n) = T(k) + T(n - k - 1) + c$$

e provare, per sostituzione, che $T(n) = an + b$ è soluzione per opportune costanti a, b .

Esercizio 1 (9 punti) Realizzare una funzione $append(T, x, z)$ che dato un albero binario di ricerca T , un suo nodo foglia x ed un nuovo nodo z , verifica se è possibile inserire z come figlio (destro o sinistro) di x in T . In caso affermativo ritorna l'albero esteso, altrimenti ritorna nil . (Assumere che i nodi dell'albero abbiano gli usuali campi $x.left$, $x.right$, $x.p$, $x.key$ e la radice sia $T.root$.) Valutarne la complessità.

Soluzione: Affinchè z possa essere inserito come figlio destro o sinistro di x , occorre in primo luogo che la chiave di z sia $\leq$ delle chiavi degli antenati di z che hanno x e quindi z nel sottoalbero sinistro e $\geq$ delle chiavi degli antenati di x che hanno x nel sottoalbero destro.

Quindi risalendo tali antenati, si verifica che la condizione sia soddisfatta. Se questo accade e $z.key$ è $\leq x.key$ potrà essere inserito come figlio sinistro di x , altrimenti come figlio destro. Altrimenti z non può essere figlio di x e si ritorna nil .

Lo pseudocodice può essere il seguente:

```
append(T,x,z)

# verifica che z.key sia <= delle chiavi degli antenati di x
# per i quali x e' nel sottoalbero dx e
y = x

while (y.p <> nil) and
    ( (y == y.p.left and z.key <= y.p.key) or
      (y == y.p.right and z.key >= y.p.key) )
    y = y.p

# se siamo arrivati alla radice, significa che la condizione e' soddisfatta
if (y.p == nil)
    # e z viene inserito come figlio dx o sx
    z.p = x
    if (z.key <= x.key)
        x.left = z
    else
        x.right = z

    return T
else
    # altrimenti si ritorna nil
    return nil
```

Append: verifichiamo se possiamo inserire "z" come figlio di "x" in T. Per farlo, poniamo il solito nodo copia di "x" nel nodo "y". Quindi: finché il nodo "y" ha un parent, continuiamo a salire l'albero fino ad arrivare il più possibile vicino al parent: questo ci assicura di inserire effettivamente il nodo "z" come figlio di "y" (infatti, "y" diventa uguale al suo parent perché continuiamo a risalire l'albero).

Se siamo arrivati alla radice, quindi non possiamo andare più in alto di lei (y.p == nil), allora il parent di "z" sarà proprio "x" e capiamo a seconda del valore della chiave, dove andare ad inserire "z", se come figlio dx o sx di "x", ritornando tutto l'albero T, altrimenti si ritorna nil

La complessità è chiaramente $O(h)$, dove h è l'altezza dell'albero. Infatti, il ciclo while effettua un numero di iterazioni pari al numero di antenati di x , e quindi limitato dall'altezza dell'albero. E ciascuna iterazione ha costo costante.

Esercizio 2 (10 punti) Dato un insieme di n numeri reali positivi e distinti $S = \{a_1, a_2, \dots, a_n\}$, con $0 < a_i < a_j < 1$ per $1 \leq i < j \leq n$, un $(2,1)$ -boxing di S è una partizione $P = \{S_1, S_2, \dots, S_k\}$ di S in k

sottoinsiemi (cioè, $\bigcup_{j=1}^k S_j = S$ e $S_r \cap S_t = \emptyset, 1 \leq r \neq t \leq k$) che soddisfa inoltre i seguenti vincoli:

$$|S_j| \leq 2 \quad \text{e} \quad \sum_{a \in S_j} a \leq 1, \quad 1 \leq j \leq k.$$

In altre parole, ogni sottoinsieme contiene al più due valori la cui somma è al più uno. Dato S , si vuole determinare un $(2,1)$ -boxing che minimizza il numero di sottoinsiemi della partizione.

1. Scrivere il codice di un algoritmo greedy che restituisce un $(2,1)$ -boxing ottimo in tempo lineare. (Suggerimento: si creino i sottoinsiemi in modo opportuno basandosi sulla sequenza ordinata.)
2. Si enunci la proprietà di scelta greedy per l'algoritmo sviluppato al punto precedente e la si dimostri, cioè si dimostri che esiste sempre una soluzione ottima che contiene la scelta greedy.

(2-1) boxing: ogni sottoinsieme contiene al più

2 valori la cui somma è al più 1. Si chiede di minimizzare il numero di sottoinsiemi della partizione.

Per fare ciò, la somma è al più 1 considerando per certo gli estremi (inferiore e superiore).

Grazie a questi, sappiamo per certo che, prendendo il valore più grande (sup) e il valore più piccolo (inf), la scelta funziona.

Soluzione:

1. L'idea è provare ad accoppiare il numero più piccolo (a_1) con quello più grande (a_n). Se la loro somma è al massimo 1, allora $S_1 = \{a_1, a_n\}$, altrimenti $S_1 = \{a_n\}$. Poi si procede analogamente sul sottoproblema $S \setminus S_1$.

(2,1)-BOXING(S)

$n \leftarrow |S|$

Inizializziamo l'insieme, la partizione e gli estremi.

$P \leftarrow \text{empty_set}$

$\text{first} \leftarrow 1$

$\text{last} \leftarrow n$

while ($\text{first} \leq \text{last}$) Si considera un ciclo per cui "first" è $\leq$ "last" (perché scansioniamo gli estremi, come detto)

if ($\text{first} < \text{last}$) **and** $a_{\text{first}} + a_{\text{last}} \leq 1$ **then**

$P \leftarrow P \cup \{a_{\text{first}}, a_{\text{last}}\}$

$\text{first} \leftarrow \text{first} + 1$

else

$P \leftarrow P \cup \{a_{\text{last}}\}$

$\text{last} \leftarrow \text{last} - 1$

return P

Esempio:

1 2 3 4 5 6 7

$P = (7, 6, 5, 4, 3, 2, 1)$

Se l'estremo inf. è $<$ dell'estremo sup.

non abbiamo ancora salvato nulla in P (non abbiamo estr. inf)

allora salvo in P sia l'estremo inf che l'estremo sup

e incremento first. Salvandolo una volta sola, so che la somma è sempre > 1 .

Altrimenti,

salvo solo l'estremo superiore migliore, la cui somma è sempre $>= 1$

Questo algoritmo scansiona ogni elemento una sola volta, quindi la sua complessità è lineare.

2. La scelta greedy è $\{a_1, a_n\}$ se $n > 1$ e $a_1 + a_n \leq 1$, altrimenti $\{a_n\}$. Ora dimostriamo che esiste sempre una soluzione ottima che contiene la scelta greedy. I casi $n = 1$ e $a_1 + a_n > 1$ sono banali, visto che in questi casi ogni soluzione ammissibile deve contenere il sottoinsieme $\{a_n\}$. Quindi assumiamo che la scelta greedy sia $\{a_1, a_n\}$. Consideriamo una qualsiasi soluzione ottima dove a_1 e a_n non sono accoppiati nello stesso sottoinsieme. Quindi, esistono due sottoinsiemi S_1 e S_2 , con $a_1 \in S_1$ e $a_n \in S_2$. Sostituiamo questi due sottoinsiemi con $S'_1 = \{a_1, a_n\}$ (cioè, la scelta greedy) e $S'_2 = S_1 \cup S_2 \setminus \{a_1, a_n\}$. $|S'_2| \leq 2$ e, se $|S'_2| = 2$, allora $S'_2 = \{a_s, a_t\}$ con $a_s \in S_1$ e $a_t \in S_2$. Siccome a_t era precedentemente accoppiato con a_n , a maggior ragione può essere accoppiato con $a_s < a_n$, quindi la nuova soluzione così creata è ammissibile e ancora ottima.

Spiegata in termini semplici: la scelta greedy consiste nel scegliere quello che sta agli estremi.

Esiste sempre una soluzione ottima in quanto il sottoinsieme che consideriamo è sempre formato da almeno

due elementi (tolto il caso base) per cui tra questi ci siano i nostri estremi inf. e sup.

Andando a fare le differenze ottime (scegliendo di volta in volta gli estremi migliori), di sicuro

avremo ancora una differenza ottima, perché scegliamo gli estremi migliori (più piccolo per inf. e più grande per sup) in ogni momento.